Design and Implementation of a Side-Channel Resistant FPGA-Based Signing Engine for Cryptocurrency Hardware Wallets

Esabu Blessing Obehi
Matriculation No. 240408054

Department of Electrical and Electronics Engineering
University of Lagos, Akoka, Lagos State, Nigeria
240408054@live.unilag.edu.ng

ABSTRACT

Cryptocurrency hardware wallets based on microcontroller units (MCUs) remain vulnerable to side-channel analysis (SCA) attacks at the cryptographic signing layer. Power traces captured during secp256k1 Elliptic Curve Digital Signature Algorithm (ECDSA) scalar multiplication on MCU platforms exhibit data-dependent switching activity that enables Differential Power Analysis (DPA) to reconstruct 256-bit private keys. This paper presents the design and implementation of a side-channel resistant signing engine targeting the MicroPhase A7-Lite FPGA (Xilinx Artix-7 XC7A100T). The central countermeasure is the Montgomery ladder algorithm for elliptic curve scalar multiplication, which enforces a constant-time, constant-operation-sequence execution path regardless of scalar bit values, providing algorithmic-level resistance to Simple Power Analysis (SPA) and DPA. Field operations are implemented in Jacobian projective coordinates to eliminate costly modular inversions. The design was synthesised and verified in Xilinx Vivado, with algorithm-level correctness validated against a Python reference implementation on the secp256k1 test vectors underlying the Bitcoin and Ethereum signing specifications. The prototype executes complete ECDSA transaction signing and was validated on a public blockchain testnet. This work demonstrates that constant-time FPGA-based signing architectures provide a structurally superior alternative to MCU software signing cores for private-key custody applications.

Keywords: FPGA, secp256k1, ECDSA, side-channel analysis, Montgomery ladder, Jacobian coordinates, hardware wallet, cryptocurrency, VHDL, Vivado, power analysis, DPA, SPA.

1. INTRODUCTION

The ownership of assets on Bitcoin and Ethereum is established exclusively by possession of cryptographic private keys; compromise of a private key constitutes an irreversible loss with no institutional recourse. Hardware wallets were introduced to address the key-custody problem by isolating private key material from internet-connected hosts. Commercial implementations — Ledger, Trezor, and their derivatives — store private keys on dedicated microcontroller units (MCUs) or Secure Elements (SEs) and execute ECDSA signing in firmware.

However, power-based side-channel analysis (SCA) attacks remain a practical threat. Ledger Donjon researchers demonstrated extraction of both the PIN and private key from a Trezor One hardware wallet using Differential Power Analysis (DPA) in 2019, requiring fewer than 30,000 power

traces [1]. In 2023, Unciphered extracted an unencrypted mnemonic seed from a OneKey wallet by intercepting inter-chip bus communication with an FPGA-based logic analyser [2]. These documented exploits establish a clear attack surface: the sequential, software-based ECDSA scalar multiplication on MCUs produces a power consumption profile whose statistical properties correlate with secret key bits.

Field-Programmable Gate Arrays (FPGAs) offer a compelling countermeasure architecture. A signing engine implemented in hardware description language (VHDL) and synthesised onto an FPGA fabric can be designed to execute a mathematically identical computation for every possible key bit, eliminating the data-dependent switching divergence that enables SPA and DPA. This paper presents such a design, targeting the MicroPhase A7-Lite development board (Artix-7 XC7A100T), synthesised with Xilinx Vivado. A Python reference model validated algorithm-level correctness prior to RTL implementation.

2. BACKGROUND AND RELATED WORK

2.1 secp256k1 ECDSA and the Scalar Multiplication Problem

The secp256k1 elliptic curve is defined by $y^2 = x^3 + 7$ over the 256-bit prime field ■_p, where p = 2²■■ – 2³² – 2■ – 2■ – 2■ – 2■ – 2■ – 1. A user's private key k is a uniformly random 256-bit integer; the corresponding public key is Q = k·G, with G the curve's base point. ECDSA signature generation requires computing the ephemeral public key R = k_e·G for a per-signature nonce k_e, followed by the computation of the signature scalar s = k_e■¹(z + r·k) mod n, where z is the transaction hash, r is the x-coordinate of R mod n, and n is the curve order [3].

Elliptic curve scalar multiplication (ECSM) — computing k·G for a 256-bit scalar k — is the computationally dominant operation. A naive double-and-add algorithm evaluates the scalar bit by bit: a '1' bit triggers a point addition, a '0' bit does not, creating a measurable difference in power consumption that Simple Power Analysis (SPA) can exploit to recover k directly from a single trace [4].

2.2 Side-Channel Analysis Attacks

Side-channel attacks exploit physical observables of a computation rather than mathematical weaknesses in the algorithm itself. The linear power model [5] states that the instantaneous power consumption of a CMOS circuit is proportional to the Hamming weight of intermediate values being processed:

$$P(t) = \alpha \cdot HW(v(t)) + \beta \cdot P_data + \varepsilon(t)$$

where HW(v(t)) is the Hamming weight of the intermediate value v at time t, α and β are device-specific constants, and $\varepsilon(t)$ is additive noise. Differential Power Analysis (DPA) [5] exploits this model statistically across many traces to recover key bits even when individual traces are too noisy for SPA. TVLA (Test Vector Leakage Assessment) [6] uses a Welch's t-test between fixed-input and random-input trace populations to detect any form of key-dependent leakage.

2.3 Existing Hardware Countermeasures

Several algorithmic countermeasures have been proposed to eliminate data-dependent execution in ECSM. The Montgomery ladder [7] ensures that every scalar bit evaluation executes exactly one point doubling and one point addition in a fixed sequence, regardless of the bit value. Projective coordinate randomisation [8] re-randomises the internal representation of intermediate points, breaking the correlation between scalar bits and Hamming weights even if the operation sequence leaks. The EthVault project [9] demonstrated full secp256k1 ECC within 27% of LUTs on a

Zynq-class device, establishing feasibility of FPGA-based signing engines.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 Target Platform

The MicroPhase A7-Lite development board hosts the Xilinx Artix-7 XC7A100T FPGA, providing 101,400 logic cells, 135,200 CLB flip-flops, 240 DSP48E1 slices, and 135 36Kb BRAM tiles. The Artix-7 family's DSP48E1 slices support efficient 25×18-bit signed multiply-accumulate operations, making them well-suited for the 256-bit modular arithmetic required by secp256k1 field operations. The board's compact form factor and standalone operation without an attached host processor replicate the deployment scenario of an embedded signing device.

3.2 Montgomery Ladder Algorithm

The Montgomery ladder for ECSM maintains two running curve points, R■ and R■, initialised to R■ = ■ (point at infinity) and R■ = G (generator). For each bit b_i of the scalar k from the most significant bit downward:

$$\textit{If } b_i = 0\textit{: } R■ \leftarrow R■ + R■\textit{; } R■ \leftarrow 2 \cdot R■$$

$$\textit{If } b_i = 1\textit{: } R■ \leftarrow R■ + R■\textit{; } R■ \leftarrow 2 \cdot R■$$

In both cases, exactly one point addition and one point doubling are executed, with the operands selected by a conditional swap that is itself implemented as a constant-time masking operation. The final value of R■ is k·G. This formulation was chosen over the non-adjacent form (NAF) or sliding-window methods precisely because its operation count is fixed at 256 iterations of {add, double}, irrespective of the Hamming weight of k [7].

3.3 Jacobian Projective Coordinates

Affine point operations require a modular inversion for each point addition, costing approximately 256 field multiplications per inversion using Fermat's little theorem. Jacobian projective coordinates represent a point (X, Y, Z) in projective space such that the affine point corresponds to (X/Z^2, Y/Z^3). Under this representation, point doubling and point addition are performed entirely with field multiplications and additions — no inversions — at the cost of three field elements per point. A single affine conversion at the end of the ladder recovers the result in (x, y) form [3].

The field arithmetic over ■_p is implemented using Montgomery modular multiplication [10], which converts the modular reduction into a sequence of right-shift operations amenable to efficient mapping onto the DSP48E1 slices of the Artix-7.

3.4 VHDL Implementation Structure

The signing engine is decomposed into three hierarchical layers, each implemented as a separate VHDL entity:

- **Field Arithmetic Unit (FAU):** 256-bit modular addition, subtraction, and Montgomery multiplication over ■_p, mapped to DSP48E1 slices. Combinational delay is managed through pipelining across registered pipeline stages.
- **Point Operation Core:** Jacobian point doubling and point addition, instantiating the FAU. Each operation is orchestrated by a finite state machine (FSM) that sequences the field multiplications, additions, and intermediate register transfers.
- **Montgomery Ladder Controller:** Top-level FSM that iterates over the 256 scalar bits, conditionally swaps R■/R■ registers using constant-time masking, and drives the point

operation core. The constant-time swap is implemented as XOR-based conditional exchange with no branch instructions.

A Python reference model implementing the identical mathematical operations — secp256k1 field arithmetic, Jacobian coordinates, and the Montgomery ladder — was developed first and used to generate test vectors for every module. RTL simulation against these vectors confirmed bit-exact agreement before synthesis.

3.5 SCA Countermeasures at the RTL Level

Three complementary countermeasures are applied to address the three levels at which leakage can occur:

(1) Algorithm level — Montgomery ladder: The fixed double-and-add sequence eliminates the conditional branch structure of naive double-and-add, preventing SPA extraction of scalar bits from a single trace.

(2) Representation level — projective coordinate randomisation: After each ladder step, the projective representation of the accumulator points is randomised by multiplying all three Jacobian coordinates by a fresh random value λ: $(X, Y, Z) \rightarrow (\lambda^2 X, \lambda^3 Y, \lambda Z)$. This is equivalent under projective equivalence but ensures that the Hamming weight of intermediate coordinates carries no information about the scalar k, defeating first-order DPA [8].

(3) Circuit level — balanced register toggling: Registers holding key material are held in dual-rail complementary encoding during idle cycles, equalising the average switching activity and reducing net power variation across clock cycles.

4. VALIDATION AND RESULTS

4.1 Python Reference Model

A reference implementation was written in Python to validate the mathematical correctness of the algorithm before VHDL coding. The reference model implements: (a) 256-bit modular arithmetic over ■_p using Python's arbitrary-precision integers; (b) Jacobian projective point doubling and point addition; (c) the Montgomery ladder for scalar multiplication; and (d) the full ECDSA signing and verification protocol per RFC 6979 deterministic nonce generation [11].

The reference model was validated against the secp256k1 test vectors published in the Bitcoin Core repository and the Ethereum Yellow Paper. ECDSA signatures generated by the reference model were accepted by both the Python ecdsa library and by web3.py's verification functions. This provided a mathematical ground truth for subsequent RTL verification.

4.2 VHDL Bring-Up and Simulation

The VHDL implementation was brought up in Xilinx Vivado using behavioural simulation. Test benches applied the secp256k1 base-point multiplication (k = 1 through k = 16) and a set of random 256-bit scalar test vectors. The RTL simulation results matched the Python reference model to 256-bit precision across all test vectors, confirming correct implementation of the Jacobian arithmetic and the Montgomery ladder control logic.

Synthesis was performed targeting the Artix-7 XC7A100T on the MicroPhase A7-Lite board using Vivado 2023.1. The critical path was identified through Report Timing Summary, with worst negative slack constrained to zero at the target clock frequency. Synthesis completed without critical warnings related to timing violations or unresolved combinational loops.

4.3 Testnet Validation

The final prototype was deployed as a standalone signing device. A transaction was constructed following the Bitcoin serialisation format (version, inputs, outputs, locktime), hashed with double-SHA256 to produce the signing message z, and submitted to the signing engine. The ECDSA signature (r, s) produced by the FPGA was serialised in DER format and broadcast to the Bitcoin testnet3 public blockchain. The transaction was accepted, mined in a subsequent block, and confirmed, validating end-to-end signing correctness from the FPGA output through the live blockchain verification process.

4.4 Side-Channel Resistance Assessment

The architectural SCA resistance of the design is evaluated analytically with respect to the three attack models:

SPA: The Montgomery ladder executes 256 iterations each containing exactly one point doubling and one point addition, with no conditional skips. A power trace of the ECSM operation therefore contains 512 point operations in a fixed pattern regardless of the scalar value, eliminating the distinctive doubling-only and doubling-plus-addition patterns that characterise the vulnerable double-and-add algorithm.

DPA: Projective coordinate randomisation ensures that the Hamming weights of intermediate values processed at each field multiplication do not carry exploitable correlation with the scalar bits. Each intermediate point is equivalently re-randomised, so an adversary collecting multiple traces cannot align intermediate values across traces to build a correlation model.

Inter-chip leakage: All sensitive operations — private key storage, nonce generation, scalar multiplication — are confined within the FPGA fabric. No private key material is transmitted off-chip. The monolithic architecture structurally eliminates the inter-chip bus interception attack demonstrated against the OneKey wallet [2].

5. DISCUSSION

The Montgomery ladder provides provably uniform operation sequences at the algorithm level. Combined with Jacobian coordinate randomisation and constant-time conditional swaps at the RTL level, the architecture addresses the three principal leakage pathways documented in the SCA literature. The testnet validation confirms that the mathematical correctness is preserved end-to-end, from Python reference model through VHDL simulation to live blockchain transaction.

A limitation of the current work is that formal SCA measurement using a ChipWhisperer platform and TVLA t-tests has not yet been conducted. Architectural countermeasures are necessary but not sufficient to guarantee first-order DPA resistance; physical measurements remain the definitive validation method. Future work will add a shunt resistor measurement point to the A7-Lite power supply rail and acquire power traces using the ChipWhisperer-Lite at 105 MS/s, applying both CPA (target: Pearson $\rho_{max} < 0.05$ over 10,000 traces) and TVLA (target: $|t| < 4.5$ at all sample points over 5,000 fixed and 5,000 random traces [6]).

Resource utilisation on the XC7A100T is well within the available fabric. The three-layer hierarchy — field arithmetic, point operations, ladder controller — maps efficiently to the DSP48E1 slices and CLB LUTs available in the Artix-7 family. This leaves substantial margin for the addition of BIP-32 hierarchical key derivation, a USB interface controller, and a secure display driver in future iterations.

6. CONCLUSION

This paper presented the design and implementation of a secp256k1 ECDSA signing engine for cryptocurrency hardware wallets, targeting the MicroPhase A7-Lite FPGA (Artix-7 XC7A100T) and synthesised in Xilinx Vivado. The Montgomery ladder algorithm provides constant-time scalar multiplication at the algorithm level; Jacobian projective coordinates eliminate costly modular inversions; projective coordinate randomisation and constant-time conditional swaps provide RTL-level countermeasures against DPA. A Python reference model validated mathematical correctness before RTL implementation, and the prototype successfully signed a transaction accepted on the Bitcoin testnet3 public blockchain. The work demonstrates that FPGA-based signing architectures provide a structurally superior — and practically realisable — alternative to MCU software signing cores for private-key custody applications.

REFERENCES

[1] Ledger Donjon, "Breaking Trezor One with SCA," Ledger Blog, 2019. [Online]. Available: https://www.ledger.com/blog/

[2] Unciphered, "OneKey hardware wallet vulnerability disclosure," 2023. [Online]. Available: https://unciphered.com/

[3] D. Hankerson, A. Menezes, and S. Vanstone, Guide to Elliptic Curve Cryptography. Springer, 2004.

[4] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in Proc. CRYPTO 1999, Lecture Notes in Computer Science, vol. 1666, pp. 388–397, Springer.

[5] S. Mangard, E. Oswald, and T. Popp, Power Analysis Attacks: Revealing the Secrets of Smart Cards. Springer, 2007.

[6] G. Goodwill, B. Jun, J. Jaffe, and P. Rohatgi, "A testing methodology for side-channel resistance validation," NIST Non-invasive Attack Testing Workshop, 2011.

[7] P. L. Montgomery, "Speeding the Pollard and elliptic curve methods of factorization," Mathematics of Computation, vol. 48, no. 177, pp. 243–264, 1987.

[8] M. Joye and C. Tymen, "Protections against differential analysis for elliptic curve cryptography," in Proc. CHES 2001, LNCS vol. 2162, pp. 377–390.

[9] Y. Lemayian et al., "EthVault: FPGA-based Ethereum hardware wallet," IEEE Access, 2025.

[10] P. L. Montgomery, "Modular multiplication without trial division," Mathematics of Computation, vol. 44, no. 170, pp. 519–521, 1985.

[11] T. Pornin, "RFC 6979: Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)," IETF RFC 6979, 2013.